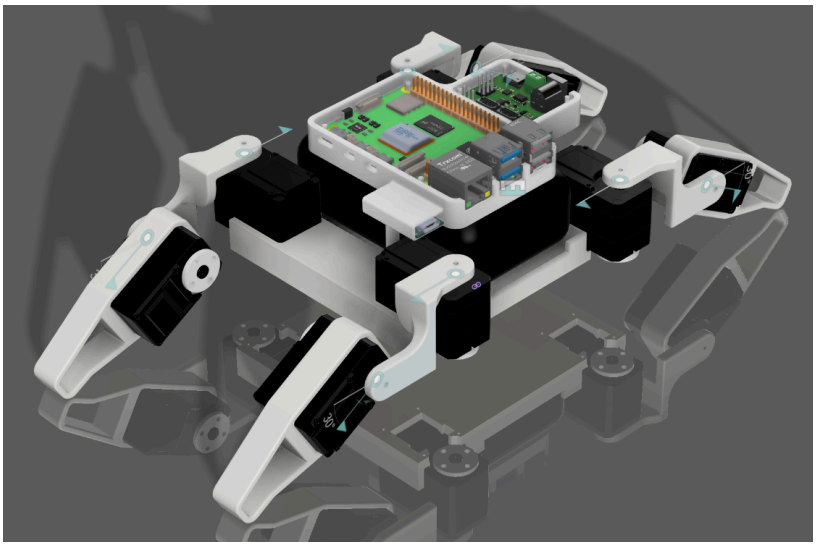
Spiderbot - Mark 1

By Yuvraj Ruparel

Overview:

Spiderbot is a four legged robot which is designed to be able to walk forward, backward, and rotate in its place, clockwise or anticlockwise, built entirely as a digital design and simulation. The robot mimics an insect's movement patterns, specifically a spider. Each leg carries two ST3215 - HS bus servos. One is used for the hip rotation about the vertical axis to sweep the leg, the other is used for the knee, rotating to lift and plant the foot. A Raspberry Pi 5 runs the control loop and drives all servos through a Waveshare Bus Servo Adapter A. The INIU 25000 mAh powerbank powers the entire robot, connected to the Raspberry Pi, and a USB PD trigger feeding the servo bus.

The complete assembly weighs 1137g, roughly 1200 with the wiring, within the 3500 g limit, including the wiring and electronic components, and occupies a total of 155 × 295 × 110 mm volume when standing, fitting in the 300 mm cube constraint. An electronics tray snaps into the slots in the base plate, forming the chassis. The tray is designed specifically to snap onto and hold the Raspberry Pi 5 board, the Servo Bus Adapter, and the USB PD trigger. Every part was designed in Autodesk Fusion, to be printed in PLA, and simulated in MuJoCo, using python and VS Code.



Design History Timeline:

1. **Mon 24 Aug:** Started by looking into how four-legged robots actually walk. Watched footage of quadruped robots, read about gait patterns, and sketched out a rough timeline for the build.
2. **Wed 26 Aug:** First concept sketch. The plan at this point was three motors per leg, which capped me at two legs under the eight-servo limit. To keep it stable I gave the two rear

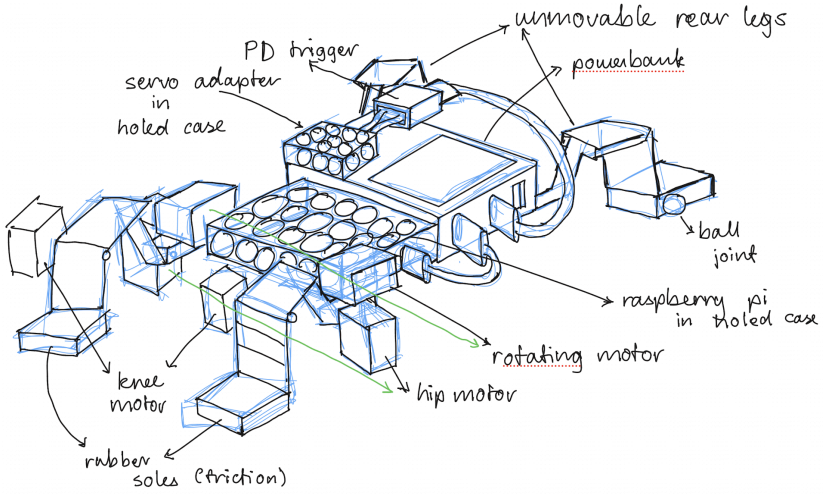
legs passive ball joints so they could roll in any direction while the front legs did the driving.

3. **Thu 27 Aug:** Rethought the movement. I found an arrangement that gets both translation and rotation out of two motors per leg, which freed up the servo budget for four driven legs. The ball joints were no longer needed and the design became a symmetric quadruped.
4. **Fri 28 Aug:** Built the full assembly in Fusion. Chassis, legs, servo mounts, and the joints between them.
5. **Sun 30 Aug:** Laid out the electronics inside the CAD and worked out the wiring: what connects to what, and how power splits between the Pi and the servo bus.
6. **Mon 31 Aug:** Learned MuJoCo from scratch. Started with a falling box to confirm the setup worked before touching the robot model.
7. **Tue 1 Sep:** Measured every joint position and offset out of Fusion and wrote the model file, defining the body tree, joint axes, ranges, and masses.
8. **Wed 2 Sep:** Ran the simulation. Swept knee angles to find a stable standing height, then tuned the gait until the robot walked forward, backward, and turned reliably.
9. **Thu 3 Sep:** Report and documentation.

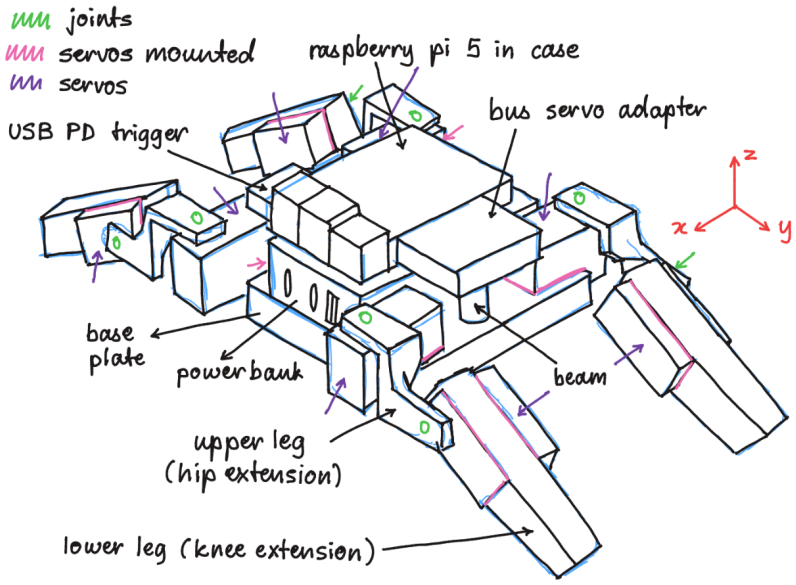
Conceptual Design:

Each leg needs to do two things, lift the foot off the ground and move it relative to the body. The knee lifts, and the hip moves the body. Both walking and turning come from those two motions. The eight servo limit shaped the design.

My first idea was inspired by a mammal walking. However, those hips had two axes, one to swing the leg back and forth, and another to turn the leg inwards or outwards. With the knee, that's three motors per leg, leaving only enough motors for 2 legs. So the first concept had two driven front legs and two rear legs with ball joints on their feet that rolled along in whatever direction the front legs dragged the body.



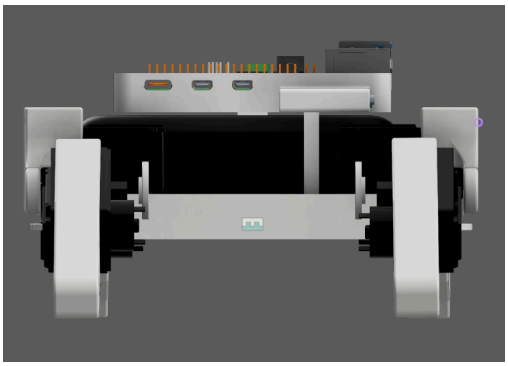
Although it would have worked, it would have been much more complex to design and would require a greater setup to simulate and optimise. The fix came from taking inspiration from insects. A spider's legs stick out sideways and sweep through the ground plan. The sweeping can not only make the robot rotate, but can create force to push the robot ahead and behind by placing the foot down and sweeping. One motor, 2 functions. That allowed me to have two motors per leg, using all eight servos, and have every leg motor driven.

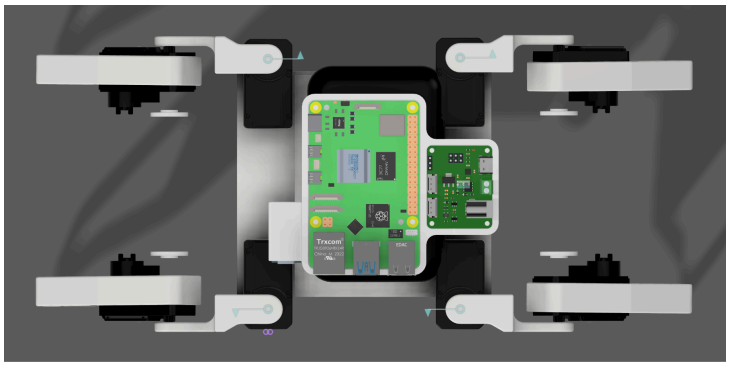


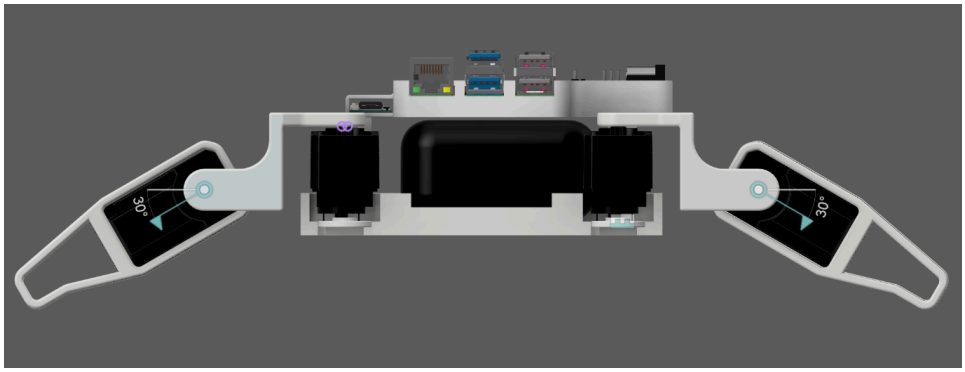
The servo mounts to the base plate with its shaft pointing up, so the leg sweeps horizontally. The knee servo sits on the lower leg with its shaft connected to the upper leg, so the lower leg swings up and down. The power bank is the heaviest part and is placed low and in the center, with the electronics tray containing the Raspberry Pi and servo adapter above it.

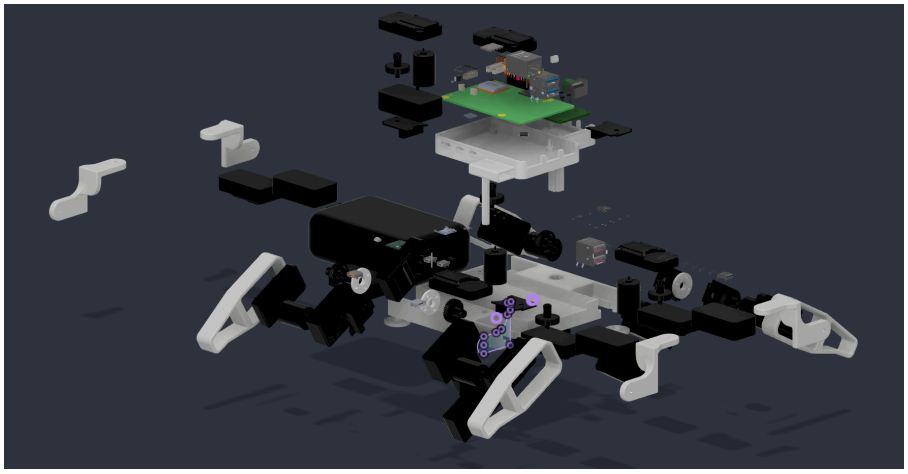
Mechanical Design:

Below are all the renders of the final assembly, including the top, front, and side views. The isometric view is shown in the overview. There is also an exploded view of the components.









Each leg has two joints. The hip rotates about the z-axis, sweeping the leg through the xy-plane. The knee rotates about the horizontal axis, which may vary between x and y based on the direction it faces, swinging the leg up and down. Since the hip shafts point up, the legs are in a sprawled stance rather than tucked under the body.

The hip can rotate up to 25° into the side of the robot, and up to 60° away from the sides, and the knee can travel from 30° to 60° below the horizontal. The mass, as shown in the table below, adds up to approximately 1200 g, including an approximate weight for the wiring.

Mass Table			
Component	**Unit Mass (g)**	**Quantity**	**Total Mass (g)**
Power bank	393	1	393
ST3215-HS servo	55	8	440
Base plate	110	1	110
Raspberry Pi 5	50	1	50
Electronics tray	42	1	42
Knee extension	12	4	48

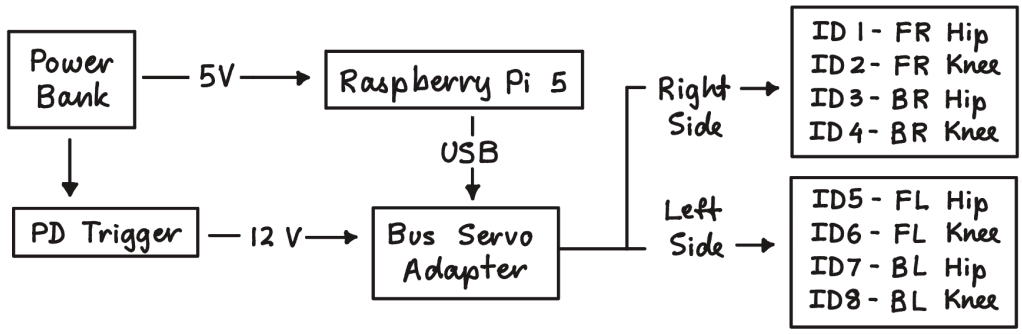
Hip extension	7	4	28
USB PD trigger	16	1	16
Bus servo adapter	10	1	10
Total			1137

The assembly splits into parts that can be printed separately and snapped into slots rather than being printed as one piece. Only two leg parts are unique, one upper and one lower, with mirrored copies rotated 180° to serve the opposite corners, so four legs come from two designs.

Electrical Designs:

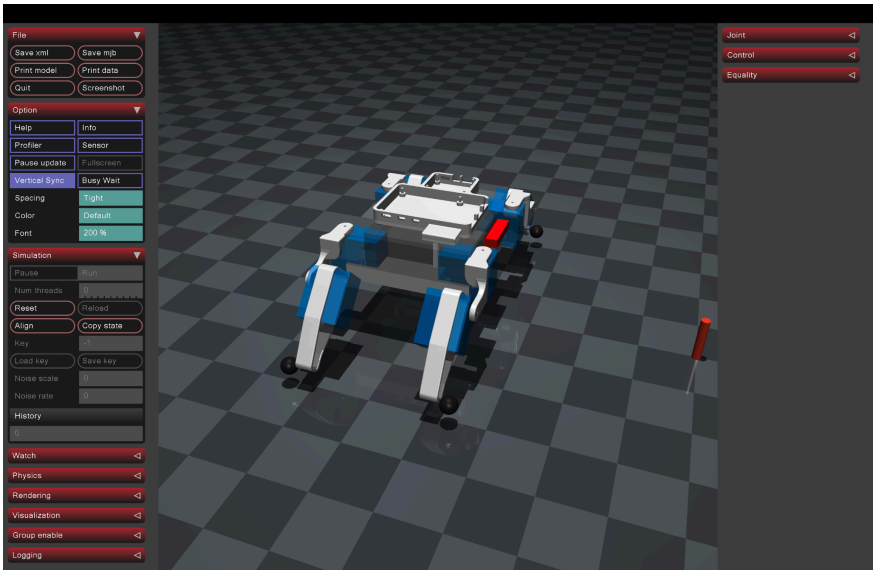
The powerbank gives power to two separate outputs. One goes straight to the Raspberry Pi 5 at 5V. The other goes through a USB PD trigger to the servo bus. Keeping them separate matters since eight servos starting together use a lot of current, and if the Raspberry Pi shared that line, it could shut down.

The Raspberry Pi connects to the bus servo adapter with a USB. The adapter converts it into a signal that the servos understand. The servos aren't wired one by one, each has a second connector that powers the next, so they chain together. I split them into two chains of four, one for each side of the robot. Each servo is numbered from 1 to 8, which is how the Raspberry Pi tells them apart on a shared line. I chose Raspberry Pi 5 over the Orange Pi 5 since it is documented better, and simpler to set up.



Simulations:

I exported each part from Fusion as an STL file, with the mesh origins placed on the joint axes so they could easily and directly be positioned for simulation. I wrote the MJCF model file by hand from measured joint offsets I collected by measuring distances in the assembly file.



The meshes are used for only visuals and do not collide. The contact happens through a small sphere at each foot, which is more stable than colliding against the printed geometry. Servos, the powerbank, and boards are modelled as simple boxes carrying their recorded masses, positioned where they actually sit, to replicate the conditions to real life. The model has nine rigid bodies, the chassis, four upper and four lower legs.

To find a standing pose I swept the knee angle from 30° to 60° and let the robot settle at each. A shallower stance pushes the feet out of the 300 mm limit. I also logged each foot's height through a full gait cycle to confirm the legs were lifting in the intended order.

Control Policy:

The gait is manually written rather than learned, since there is no hardware to run a trained policy on, and manually written gait is easy to check against what the robot actually does. Every leg follows the same cycle just starting at a different point, with half the cycle in the air, half on the ground. This way two legs are always lifting while the other two push.

The knee lifts the leg, the hip swings it forward and the knee lowers the foot. Then the foot stays planted and the hip pulls back, which moves the body in front. All joints are controlled with a target angle. How hard the joint pushes towards the target is kept low so the legs move smoothly instead of snapping and the Spiderbot losing balance.

Since each hip moves about a vertical axis, the foot follows a curved path. This helps in turning, but for walking straight the curves work against each other, so the legs move in diagonally opposite pairs instead. In eight seconds the robot moved 235 mm forward and 240 mm backward, and the body was at a steady 71 mm height throughout.

Future Plans:

What I have built so far is Mark 1. The project continues in four stages from here.

Mark 2: Physical Build and Learned Gait

- Print the parts and assemble the hardware
- Replace the scripted gait with a policy trained through reinforcement learning
- Add inverse kinematics so I can command foot position in 3D space instead of joint angles, which fixes the sideways scrub and gives control over stride length and foot clearance

Mark 3: Sensing and Custom Electronics

- Design a PCB that sits on the Raspberry Pi and handles power, servo bus connectors, and sensors in one board instead of loose wiring
- Add an IMU for orientation, which fixes the heading drift measured in simulation and gives the learned policy the feedback it needs
- Add time of flight sensors on each face for obstacle detection, and one facing down to stop it walking off a table edge
- Read servo load and temperature off the bus, which needs no extra hardware
- Move the software onto ROS 2

Mark 4: Wireless Control and Audio

- Drive the robot from a phone or a game controller over Wi-Fi and Bluetooth
- Add a speaker for audio output
- Add a screen showing status while walking, and running small games when parked
- Lay out the Mark 3 PCB with these connections already in place so nothing needs redesigning

Mark 5: Camera and Voice

- Gesture recognition, so it waves back, follows the direction my finger turns, and walks towards me or away on command
- Microphone input with spoken replies through the speaker
- Stretch goal, placing two legs on my open palm and pushing up while the back legs stay planted, which needs the IMU in a tight feedback loop and may not work